517321 PRINCIPLES OF PROGRAMMING LANGUAGES

RUST TUTORIAL PRESENTATION

กลุ่ม [21] — [Smart Pointer]

1. [วีรภัทร พิริยะสถิต] — Concept + Short Code Illustration
2. [อาธารดา พรหมแทนสุด] — Detailed Code + Live Demo
3. ~~[ชื่อ สกุล] — Rust vs Other Language + PPL Analysis~~
4. ~~[ชื่อ สกุล] — Exercises + Common Mistakes + Challenge~~

คณะวิทยาศาสตร์ มหาวิทยาลัยศิลปากร | ภาคการศึกษาที่ 1/2569

PART 1 · MEMBER 1

[Smart Pointer] คืออะไร?

- [**Smart Pointer** เป็นตัวชี้รูปแบบพิเศษที่จะเข้ามาจัดการ**life cycle** ของข้อมูลและช่วยจัดการ**Memory**ได้ดั่งใจนึกแต่ก็ต้องแลกมาด้วย **ขนาด**บนMemory**ที่เพิ่มขึ้น**มากกว่าตัวชี้ทั่วไปด้วยเช่นกัน]

- [เนื่องจากRustมีขีดจำกัดว่าต้องทราบขนาดของตัวแปรและต้อง **ไม่มีการเปลี่ยนค่าโดยเด็ดขาด**(**Immutable**) Smart Pointer เข้ามาทำลายขีดจำกัดของภาษาRustที่ต้องกำหนดนู้นนี่ให้ชัดเจน โดยทำให้การ**เปลี่ยนแปลง ค่า ขนาด** หรือ **สิทธิ์การเป็นเจ้าของ**เกิดขึ้นได้**เป็นข้อยกเว้น**และนั้นคือความสามารถหลักของSmart pointer]

- [**Smart Pointer** จะมี**3ประเภท**หลักให้เลือกใช้

- 1.**Box<T>**เปลี่ยนการจองพื้นที่จากstackไปไว้บนheap

- 2.**Rc<T>**อนุญาติให้หลายส่วนในโปรแกรมเข้าถึงค่าภายในนี้ และมีระบบกัน**data race**ไปในตัว

- 3.**Refcell<T>**ใช้เปลี่ยนค่าImmutableขณะruntime]

```
use std::mem::size_of;
use std::cell::RefCell;

fn main() {

    println!("=== Size Comparison of Stack ===");
    let raw_data: i32 = 42;
    println!("Size of i32 :  {} bytes", size_of::<i32>());
    let reference: &i32 = &raw_data;
    println!("Size of &i32 (Normal Pointer): {} bytes", size_of::<&i32>());
    let ref_cell: RefCell<i32> = RefCell::new(42);
    println!("Size of RefCell<i32> (Smart Pointer): {} bytes", size_of::
<RefCell<i32>>());
}
```

ผลลัพธ์ที่คาดหวัง (Expected Output)

```
=== Size Comparison of Stack ===
Size of i32 :  4 bytes
Size of &i32 (Normal Pointer): 8 bytes
Size of RefCell<i32> (Smart Pointer): 16 bytes
```

PART 1 · MEMBER 1

[Box<T>] ใช้ตอนไหน?

- Box<T> เมื่อต้องการย้ายข้อมูลจาก Stack ไปเก็บไว้บน Heap โดยยังคงระบุความเป็นเจ้าของ **(Ownership)** ไว้ที่ตัวแปรตัวเดียวควรใช้ Box<T> เมื่อเจอสถานการณ์ดังต่อไปนี้

- 1.เมื่อต้องการสร้างชนิดข้อมูลแบบ**Recursive**
  Rust จำเป็นต้องรู้ขนาดที่แน่นอนของทุกข้อมูลในตอนคอมไพล์หากสร้างโครงสร้างข้อมูลRecursive เช่น **Linked List** หรือ **Tree** คอมไพเลอร์จะไม่สามารถคำนวณขนาดได้เนื่องจากมันจะวนลูปไปไม่สิ้นสุด

- 2. เมื่อต้องการใช้ **Trait Objects** หากต้องการเก็บค่าหลายตัวที่คนละ Type กัน แต่อยากให้พวกมันImplement ตาม Trait เดียวกัน ต้องใช้คำสั่ง **dyn Trait** ทว่า Rust จะไม่รู้ขนาดของ object นั้นล่วงหน้าจึงต้อง "ครอบ" มันด้วย Box เพื่อให้ได้ **Pointer** ที่มีขนาดแน่นอน

- 3.**Stack**มีพื้นที่จำกัดหากขนาดของ**Array&Struct****ใหญ่เกิน**อาจทำให้เกิด **Stack Overflow** การเปลี่ยนไปใช้ **Box::new(large_data)** จะเก็บข้อมูลจริงไว้บน Heap และส่งต่อเพียง Pointer เล็กๆ บน Stack เท่านั้น

```
fn main() {
    let stack_number = 5;
    let heap_pointer = Box::new(10);
    let sum = stack_number + *heap_pointer;
    println!("Total is: {}", sum);
}
```

ผลลัพธ์ที่คาดหวัง (Expected Output)

```
Total is: 15
```

PART 1 · MEMBER 1

[Rc<T>] ใช้ตอนไหน?

- **Rc<T>** หรือ (**Reference Counted**) ถูกหยิบมาใช้เมื่อต้องการแชร์ความเป็นเจ้าของข้อมูลบน Heap (**Shared Ownership**) ให้กับโค้ดหลาย ๆ ส่วนในโปรแกรม โดยทำงานอยู่ภายใต้ **Thread เดียวกันเท่านั้น**

- 1.ต้องการ **Multiple Owners**: มีเลเยอร์หรือโครงสร้างข้อมูลหลายส่วนที่จำเป็นต้องเป็น**เจ้าของข้อมูลชุดเดียวกันพร้อม ๆ กัน** และระบบไม่สามารถรับรู้ได้ล่วงหน้าตอนคอมไพล์ว่าโค้ดส่วนไหนจะใช้งานข้อมูลนั้น **เสร็จเป็นคนสุดท้าย**

- 2.ทำงานใน **Single-Thread** เท่านั้น: Rc<T> ไม่มีความปลอดภัยในเรื่อง Thread ตัวนับเป็นเพียงตัวเลขธรรมดา **ไม่ใช่ Atomic** หากพยายามส่งข้าม Thread ตัวคอมไพเลอร์ของ Rust จะ**บล็อก**ทันที

- 3.ต้องการอ่านข้อมูลอย่างเดียว (**Read-Only by default**): Rc<T> ยอมให้แชร์เฉพาะ **Immutable Reference** (&T) เท่านั้น หากต้องการแก้ไขข้อมูลภายในด้วย จำเป็น**ต้องใช้คู่กับ RefCell<T>**

```rust
use std::rc::Rc;

fn main() {
    let original = Rc::new(String::from("Shared Text"));
    {
        let owner_two = Rc::clone(&original);

        println!("Active owners inside block: {}", Rc::strong_count(&original));
        println!("Owner two sees: {}", owner_two);
    }
    println!("Active owners after block: {}", Rc::strong_count(&original));

}
```

ผลลัพธ์ที่คาดหวัง (Expected Output)

```
Active owners inside block: 2
Owner two sees: Shared Text
Active owners after block: 1
```

PART 1 · MEMBER 1

[Refcell<T>] ใช้ตอนไหน?

- **RefCell<T>**ถูกใช้เมื่อการสิทธิ์ในการแก้ไขข้อมูลจากภายใน (**Interior Mutability**) บนสภาพแวดล้อมแบบ **Single-thread** โดยปกติแล้วRust จะบังคับใช้**กฎการยืม**ข้อมูล (Borrowing Rules) ณ เวลาคอมไพล์ซึ่งหมายความว่าหาก**ทำผิดกฎ** (เช่น **พยายามยืมข้อมูลแบบแก้ไขได้ในขณะที่มีการยืมตัวอื่นค้างอยู่**) โปรแกรมของคุณจะ **Panic** (**crash**) ทันทีแทนที่จะแจ้งเตือนตอนคอมไพล์

- 1.การเป็นเจ้าของข้อมูล**ร่วมกัน**แบบ**แก้ไขได้**(**Rc<RefCell<T>>**) หากจำเป็นต้องให้โค้ดหลายส่วนเป็นเจ้าของข้อมูลชุด เดียวกันและสามารถแก้ไขมันได้ด้วยเลยต้องรวมพวกมันเข้าด้วยกัน

- 2. ความไม่เปลี่ยนแปลงภายนอก แต่เปลี่ยนข้างใน (เช่น การทำ Caching) บางครั้งออบเจกต์อาจจะดูเหมือนไม่มีการแก้ไขใด ๆ จากมุมมองภายนอก แต่ในแง่ประสิทธิภาพมันจำเป็นต้องปรับเปลี่ยนค่าภายในตัวมันเอง **ตัวอย่าง**: โครงสร้างแบบ **Cache**หรือ**LazyEvaluator** เมื่อมองจากภายนอก การเรียกใช้**.get_data(&self)** **ไม่ควร**จะไป**เปลี่ยนแปลงโครงสร้างภายนอก** แต่ในทางปฏิบัติ มันอาจจะต้องการเข้าไปเติมค่าในฟิลด์ที่เป็น Option<T> ที่ว่างอยู่เมื่อถูกเรียกใช้งานครั้งแรก

```
use std::cell::RefCell;

fn main() {
    let safe_data = RefCell::new(50);

    {
        let mut temporary_writer = safe_data.borrow_mut();
        *temporary_writer += 25;

    }
    println!("Final value: {}", safe_data.borrow());
}
```

ผลลัพธ์ที่คาดหวัง (Expected Output)

```
Final value: 75
```

PART 2 · MEMBER 2

Detailed Code + Live Demo

```
#[derive(Debug)]
enum List {
 Cons(i32, Box<List>),
 Nil,
}
use List::{Cons, Nil};

fn sum_list(list: &List) -> i32 {
 match list {
 Cons(value, next) => value + sum_list(next),
 Nil => 0,
 }
}
fn main() {
 let list = Cons(1, Box::new(Cons(2,Box::new(Cons(3,Box::new(Nil))))));
 let boxed_number = Box::new(42);
 println!("Boxed number = {}", boxed_number);
}
```

PART 2 · MEMBER 2

Detailed Code + Live Demo

```
use std::cell::RefCell;
use std::rc::Rc;

#[derive(Debug)]
struct Counter {
 value: i32,
}

fn increment(shared_counter:
&Rc<RefCell<Counter>>) {
 let mut counter = shared_counter.borrow_mut();
 counter.value += 1;
}
```

```
fn main() {
 let shared_counter = Rc::new(RefCell::new(Counter
{ value: 0 }));

 let counter_a = Rc::clone(&shared_counter);
 let counter_b = Rc::clone(&shared_counter);

 increment(&counter_a);
 increment(&counter_b);
 increment(&shared_counter);

 println!("Final value = {}",
shared_counter.borrow().value);
 println!("Total owners (strong_count) = {}",
Rc::strong_count(&shared_counter));
}
```

PART 3 · MEMBER 3

Rust vs [Other Language]

Aspect	Rust	[Other Language]
Syntax	[อธิบาย]	[อธิบาย]
Semantics / Behavior	[อธิบาย]	[อธิบาย]
Type System	[อธิบาย]	[อธิบาย]
Memory Management	[อธิบาย]	[อธิบาย]
Safety	[อธิบาย]	[อธิบาย]

Analysis: [อธิบายความแตกต่างที่สำคัญและเหตุผลด้านการออกแบบภาษา]

PART 3 · MEMBER 3

PPL Perspective — มุมมองเชิง Programming Language

Syntax

[Topic นี้เกี่ยวข้องกับ syntax อย่างไร]

Semantics

[คำสั่ง/construct มีความหมายอย่างไร]

Type System

[เกี่ยวข้องกับ type system อย่างไร ถ้ามี]

Memory / Resource

[เกี่ยวข้องกับ memory management อย่างไร ถ้ามี]

Abstraction

[scope, binding, paradigm ที่เกี่ยวข้อง]

Why Rust?

[เพิ่ม safety/reliability/performance อย่างไร]

PART 4 · MEMBER 4

Common Mistakes — ข้อผิดพลาดที่พบบ่อย

❌ **Incorrect**

```
// [โค้ดที่ผิด / ตัวอย่างข้อผิดพลาด]
fn main() {

}
```

✅ **Correct**

```
// [โค้ดที่แก้ไขถูกต้อง]
fn main() {

}
```

ทำไมถึงผิด? (Why?)

[อธิบายสาเหตุที่ทำให้เกิดข้อผิดพลาดนี้ และหลักการของ Rust ที่เกี่ยวข้อง]

PART 4 · MEMBER 4

Challenge — คำถามท้าทายผู้ฟัง

โจทย์ (Problem)

[เขียนโจทย์แบบฝึกหัด/คำถามท้าทายที่เกี่ยวข้องกับหัวข้อ ให้ผู้ฟังลองตอบสด ๆ]

💡 **Hint**

[คำใบ้]

```
// Solution
fn main() {
    // [เฉลย]
}
```

Key Takeaways — สรุปสิ่งที่ควรจำ

1. Smart Pointer ไม่เหมือน Pointer ปกติ เพราะมันเป็นคนถือครองไม่ใช่คนชี้ข้อมูล

2. 3 รูปแบบหลักมี Box<T>ที่จองพื้นที่บนheap , Rc<T>ทำให้มีคนถือข้อมูลได้หลายคน, Refcell<T>ทำให้แก้ข้อมูลimmutableได้ตอนRuntime

3. เหมาะกับงานประเภทที่ไม่ทราบขนาดงานที่แน่ชัด หรือ งานที่ต้องแก้ไขข้อมูลimmutable เช่น Recursive,Tree เป็นต้น

4. นอกจากเป็นภาษาRustเหมือนกับหัวข้ออื่นๆ หัวข้อนี้เปรียบเสมือนเครื่องมือที่จะช่วยในการเขียนภาษาRustให้สะดวกยิ่งขึ้น

References — เอกสารอ้างอิง

- https://doc.rust-lang.org/book/ch15-00-smart-pointers.html

- https://dev.to/leapcell/rust-smart-pointers-explained-ownership-memory-and-safety-1ek3

- https://stackoverflow.com/questions/55075458/understand-smart-pointers-in-rust

- https://hackmd.io/@0xdeveloperuche/ryK-pIHyee

GitHub Repository: [https://github.com/670710653/Smart-Pointer-Resource-Management]

Questions & Answers

กลุ่ม [21] — [Smart Pointer]